

# Lab 4: JMS vs Kafka

## Team Members

| Name                                      | ID              |
|-------------------------------------------|-----------------|
| <u>عمر هاني ابراهيم بشر</u>               | <u>21010891</u> |
| <u>محمد حسن محمد قباري حسن عبد الجواد</u> | <u>21011115</u> |
| <u>محمد محمد ابراهيم علي حسن</u>          | <u>21011211</u> |
| <u>نيره ابراهيم احمد محمد الخياط</u>      | <u>21011504</u> |

### Definitions:

1. **Response Time:** The total time it takes from when a request is sent until the response is received. This includes the processing time and any delays in the system.
2. **Throughput:** The rate at which data is processed or transmitted over a system. Typically, it's measured as the number of records or amount of data (in MB/sec) processed within a given time period (records per second).
3. **Latency:** The time taken for a single record or request to travel from the sender to the receiver and be processed. It is often measured in milliseconds (ms).

## Kafka Part

- Performance aspects

- Response time for both the produce and consume API calls

1<sup>st</sup> try

[illegible]

2<sup>nd</sup> Try

[illegible]

- Maximum Throughput for both produce and consume API calls.

- For producer Maximum Throughput was 147605.7 records/sec (140.77 MB/sec)

```
[appuser@6b8460e90a90 bin]$ ./kafka-producer-perf-test --topic test-topic --num-records 1000000 --record-size 1000 --throughput 100000 --producer-props bootstrap.servers=localhost:9092
148049 records sent, 29609.8 records/sec (28.24 MB/sec), 864.8 ms avg latency, 1181.0 ms max latency.
248048 records sent, 48169.6 records/sec (45.94 MB/sec), 691.4 ms avg latency, 1080.0 ms max latency.
275984 records sent, 55196.8 records/sec (52.64 MB/sec), 555.8 ms avg latency, 3484.0 ms max latency.
1000000 records sent, 59308.463318 records/sec (56.56 MB/sec), 448.85 ms avg latency, 3484.00 ms max latency, 160 ms 50th, 1100 ms 95th, 3460 ms 99th, 3481 ms 99.9th.
[appuser@6b8460e90a90 bin]$ ./kafka-producer-perf-test --topic test-topic --num-records 1000000 --record-size 1000 --throughput -1 --producer-props bootstrap.servers=localhost:9092
740390 records sent, 147605.7 records/sec (140.77 MB/sec), 100.4 ms avg latency, 758.0 ms max latency.
1000000 records sent, 115247.205255 records/sec (109.91 MB/sec), 186.43 ms avg latency, 3104.00 ms max latency, 53 ms 50th, 1305 ms 95th, 2810 ms 99th, 3181 ms 99.9th.
[appuser@6b8460e90a90 bin]$
```

-For consumer, consumer processed **170.2G07 MB/sec** of data, with a rate of **178562.7342 messages/sec**.

```
[appuser@6b8460e90a90 bin]$ ./kafka-consumer-perf-test \
> --topic test-topic \
> --messages 1000000 \
> --bootstrap-server localhost:9092
start.time, end.time, data.consumed.in.MB, MB.sec, data.consumed.in.nMsg, nMsg.sec, rebalance.time.ms, fetch.time.ms, fetch.MB.sec, fetch.nMsg.sec
2025-04-26 12:07:06:899, 2025-04-26 12:07:12:502, 954.1388, 170.2907, 1000487, 178562.7342, 4524, 1079, 884.2806, 927235.4032
[appuser@6b8460e90a90 bin]$
```

- Median latency (time between production and consumption of messages)

These are 2 sample runs for the median

Median Latency: 2.58229811 ns

Median Latency: 2.7032005E8 ns

Median Latency: 2.5822981149E10 ns

- Usability

- Number of steps needed to set up the tool

2 steps (using docker)

- Created a docker-compose.yml file with **Zookeeper + Kafka** services.
- Ran docker-compose up -d in cmd using wsl

```

C:\Users\PC\Documents\2nd term 3rd year\DDIA\Labs\Lab 4>wsl
root@DESKTOP-MU53DP6:/mnt/c/Users/PC/Documents/2nd term 3rd year/DDIA/Labs/Lab 4# systemctl start docker
root@DESKTOP-MU53DP6:/mnt/c/Users/PC/Documents/2nd term 3rd year/DDIA/Labs/Lab 4# systemctl enable docker
root@DESKTOP-MU53DP6:/mnt/c/Users/PC/Documents/2nd term 3rd year/DDIA/Labs/Lab 4# docker --version
Docker version 26.1.3, build 26.1.3-0ubuntu1-24.04.1
root@DESKTOP-MU53DP6:/mnt/c/Users/PC/Documents/2nd term 3rd year/DDIA/Labs/Lab 4# docker-compose up -d
WARN[0000] /mnt/c/Users/PC/Documents/2nd term 3rd year/DDIA/Labs/Lab 4/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] Running 18/19
   kafka Pulled                                257.6s
   63c5c6ac4b82 Pull complete                  8.2s
   ed994aa0c3a0 Pull complete                  231.9s
   zookeeper Pulled                            257.7s
   f8ec8143b82b Pull complete                  232.6s
   21b675891935 Pull complete                  230.9s
   a53d461e9e28 Pull complete                  72.7s
   7810caa94883 Pull complete                  230.3s
   a6c9c7400f0f Pull complete                  8.3s
   abbc20366061 Pull complete                  229.9s
   6db5499821dc Pull complete                  232.1s
   0c03fc0a30d2 Pull complete                  72.6s
   0e7bc823ab6b Pull complete                  231.0s
   8ea5736d6359 Pull complete                  8.3s
   4f43f96556fd Pull complete                  230.4s
   a4009f778eb Pull complete                  230.5s
   e336f26c7835 Pull complete                  229.6s
   8e22e90f45bd Pull complete                  230.1s
   bc83d6c327c5 Extracting 11 s                232.8s
[+] Running 3/3
   Network lab4_default Created                0.0s
   Container lab4-zookeeper-1 Started           7.0s
   Container lab4-kafka-1 Started              1.4s
root@DESKTOP-MU53DP6:/mnt/c/Users/PC/Documents/2nd term 3rd year/DDIA/Labs/Lab 4#

```

- Time to set up project dependencies and get a “Hello World” project working  
3 steps
- open <https://start.spring.io/>
- Chose (Project = Maven, Language = Java, Spring boot = 3.4.5, Packaging = Jar, Java=17, Dependencies = {Spring Web, Spring for Apache Kafka})
- generated C extracted the project

```

Project: springboot-kafka-DDIA-Lab4
src/main/java/net.javaguides.springboot_kafka_DDIA_Lab4/SpringbootKafkaDdiaLab4Application.java
public class SpringbootKafkaDdiaLab4Application {
    public static void main(String[] args) {
        SpringApplication.run(SpringbootKafkaDdiaLab4Application.class, args);
        System.out.println("Hello World");
    }
}

Run: SpringbootKafkaDdiaLab4Application
[INFO] 2025-04-26T14:22:18.516+03:00 [springboot-kafka-DDIA-Lab4] [main] n.j.s.SpringbootKafkaDdiaLab4Application : Starting SpringbootKafkaDdiaLab4Application
[INFO] 2025-04-26T14:22:18.520+03:00 [springboot-kafka-DDIA-Lab4] [main] n.j.s.SpringbootKafkaDdiaLab4Application : No active profile set, falling back to 1
[INFO] 2025-04-26T14:22:19.259+03:00 [springboot-kafka-DDIA-Lab4] [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat initialized with port 8080 (http)
[INFO] 2025-04-26T14:22:19.271+03:00 [springboot-kafka-DDIA-Lab4] [main] o.apache.catalina.core.StandardService : Starting service [Tomcat]
[INFO] 2025-04-26T14:22:19.271+03:00 [springboot-kafka-DDIA-Lab4] [main] o.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/10
[INFO] 2025-04-26T14:22:19.312+03:00 [springboot-kafka-DDIA-Lab4] [main] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring embedded WebApplication
[INFO] 2025-04-26T14:22:19.313+03:00 [springboot-kafka-DDIA-Lab4] [main] w.s.c.ServletWebServerApplicationContext : Root WebApplicationContext: initialization
[INFO] 2025-04-26T14:22:19.677+03:00 [springboot-kafka-DDIA-Lab4] [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port 8080 (http) with e
[INFO] 2025-04-26T14:22:19.683+03:00 [springboot-kafka-DDIA-Lab4] [main] n.j.s.SpringbootKafkaDdiaLab4Application : Started SpringbootKafkaDdiaLab4Application
Hello World

```

- Degree of code cluttering to call produce and consume APIs

## ○ 1. Produce Operation

| Metric              | Response Time Code                                                                                | Median Latency Code                                                     |
|---------------------|---------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------|
| Lines of Code (LoC) | ~10 lines                                                                                         | ~3 lines                                                                |
| Key API Calls       | - kafkaTemplate.send() (inside nested loop)<br>- KafkaTemplate injection<br>- Loop logic included | - kafkaTemplate.send()<br>- KafkaTemplate injection                     |
| Details             | Nested loop: 1,000 × 1,000 messages, each wrapped in send(). Includes timing logic.               | 10,000 messages sent with timestamp as value. Simpler, no nested loops. |

-  **Verdict:** Median latency code is less cluttered and more concise for producing.

## ○ 2. Consume Operation

| Metric              | Response Time Code                                                                                             | Median Latency Code                                                                             |
|---------------------|----------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| Lines of Code (LoC) | ~20 lines                                                                                                      | ~15 lines                                                                                       |
| Key API Calls       | - new KafkaConsumer()<br>- subscribe()<br>- poll()<br>- Loop with timestamp logic<br>- record.timestamp() used | - new KafkaConsumer()<br>- subscribe()<br>- poll()<br>- Timestamp extracted from record.value() |
| Details             | Custom config props, polling, per-record timestamp diff, median calc                                           | Similar structure but shorter logic (no nested loops, simple latency calc)                      |

-  **Verdict:** Median latency code is slightly less cluttered for consuming as well.

### ○ 3. Summary Table

| Operation | Metric        | Response Time Code | Median Latency Code | Less Cluttered                                                                             |
|-----------|---------------|--------------------|---------------------|--------------------------------------------------------------------------------------------|
| Produce   | Lines of Code | ~10 (nested loop)  | ~3                  |  Median |
|           | API Calls     | 2-3                | 2                   |  Median |
| Consume   | Lines of Code | ~20                | ~15                 |  Median |
|           | API Calls     | 3-4                | 3                   |  Median |

- The median latency setup exhibits lower code cluttering in terms of both line count and Kafka API calls used.

### ● Integrations

- It is a message that is written at the first page of kafka “Kafka’s out-of-the-box Connect interface integrates with hundreds of event sources and event sinks including Postgres, JMS, Elasticsearch, AWS S3, and more.”
- Also, it is mentioned in the documentation “Kafka community already provides hundreds of ready-to-use connectors.”
- Integrations:
  - PostgreSQL - Relational Database
    - Configure Kafka Connect by setting up the JDBC Source Connector and Kafka connect [1] which is just a configuration file.
  - Elasticsearch
    - Using elasticsearch-sink-connector which is fully automated data ingestion and indexing without requiring additional coding. [4]
    - In this article, the writer wrote 3 different ways to connect them together and one of them was elasticsearch-sink-connector. [6]
  - JMS
    - Using jms-sink and kafka-connect with different schema types [8].
  - Hadoop
    - Using Kafka Connect HDFS Sink Connector [9]
  - Cassandra

- Using Cassandra Sink [10].
- Normal file
  - This has been explained easily in kafka quick start tutorial. [5]
- Connect to almost everything using kafka connect [3]
- There is also an option to develop your own connector which is like creating a custom connect that can be uploaded to the community. [7]

## **Resources:**

1. [Connecting Kafka to PostgreSQL: Best Practices and Tips - RisingWave: Open-Source Streaming Database](#)
2. [PostgreSQL Kafka Connector: Features, Setup, and Limitations](#)
3. [Apache Kafka](#)
4. [How to ingest data to Elasticsearch through Kafka - Elasticsearch Labs](#)
5. [Apache Kafka](#)
6. [How To Send Data From Kafka To Elasticsearch + 3 Examples | Estuary](#)
7. [Apache Kafka](#)
8. [macronova/kafka-connect-jms: Sink and source capabilities to transfer messages between JMS server and Kafka brokers](#)
9. [Leveraging Kafka, Hadoop, and Spark:- A Comprehensive Guide to Big Data Streaming - Scaler Topics](#)
10. [How to Seamlessly Connect Kafka and Cassandra? \(Code Included\)](#)

**JMS**



## A) Performance Comparison

### 1- Maximum Throughput:

#### - Publisher:

- Code:

```
ExecutorService executor = Executors.newFixedThreadPool(threadCount);

ActiveMQConnectionFactory factory = new ActiveMQConnectionFactory(brokerUrl);
final String text = "\0".repeat(count: 1024) ;

long startTime = System.currentTimeMillis();

for (int i = 0; i < threadCount; i++) {
    executor.execute(() -> {
        try {
            Connection connection = factory.createConnection();
            connection.start();

            Session session = connection.createSession(b: false, Session.AUTO_ACKNOWLEDGE);
            Destination destination = session.createQueue(queueName);
            MessageProducer producer = session.createProducer(destination);
            producer.setDeliveryMode(DeliveryMode.NON_PERSISTENT);

            TextMessage message = session.createTextMessage(text);

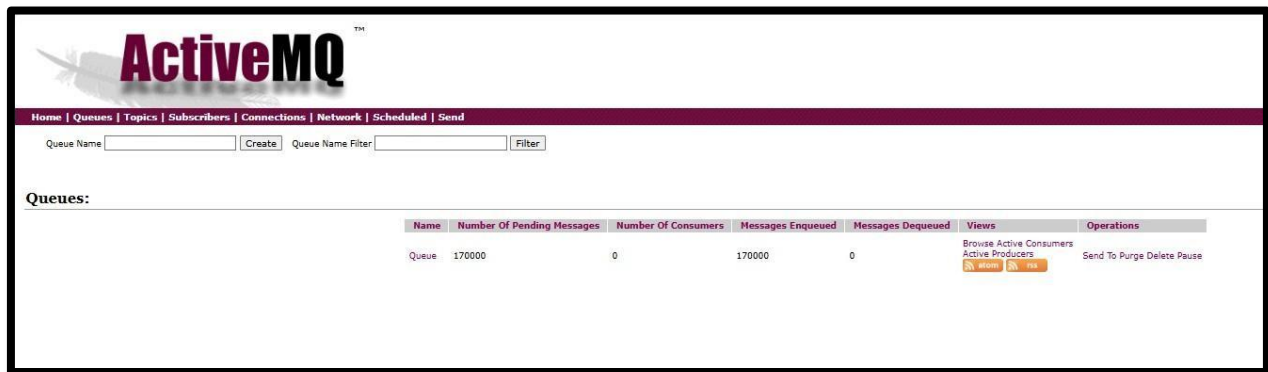
            for (int j = 0; j < messagesPerThread; j++) producer.send(message);

            session.close();
            connection.close();
        } catch (Exception e) {
            System.err.println("Error in producer thread: " + e.getMessage());
            e.printStackTrace();
        }
    });
}

executor.shutdown();
executor.awaitTermination(timeout: 5, TimeUnit.MINUTES);

long duration = System.currentTimeMillis() - startTime;
double throughput = (totalMessages * 1000.0) / duration;
```

### ○ ActiveMQ Queue:



### ○ Throughput:

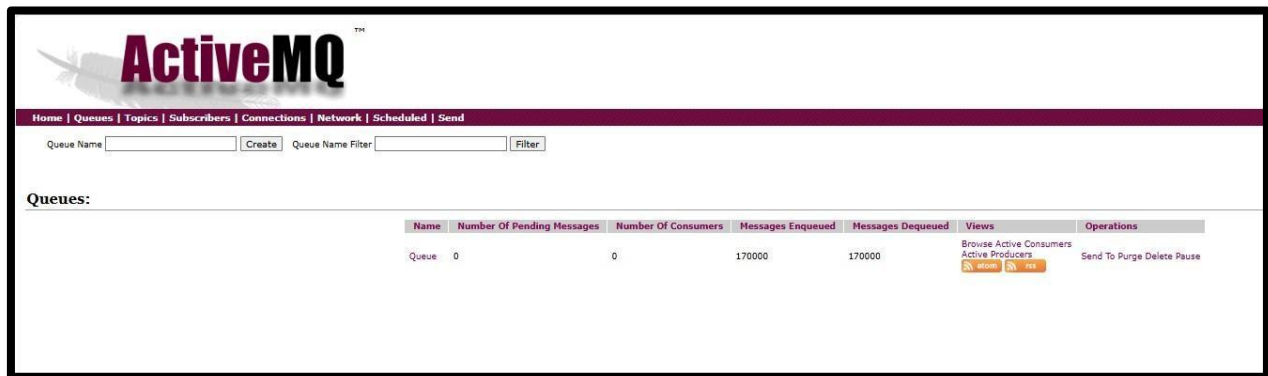
```
Producer finished.  
Total messages sent: 170000  
Time taken: 2209 ms  
Achieved throughput: 76957.90 messages/second
```

### ○ Configurations:

- **Number of threads:** Number of Available CPUs
- **Number of Messages:** 170,000.
- **Delivery Mode:** NON\_PERSISTENT.
- **Use ThreadPool.**

## - Subscriber:

- **ActiveMQ Queue:**



- **Throughput:**

```
✓ Consumer finished.  
Total messages received: 170000  
Time taken: 5477 ms  
Achieved throughput: 31038.89 messages/second
```

- **Configurations:**

- **Number of threads:** Number of Available CPUs
- **Number of Messages:** 170,000.
- **Use ThreadPool.**

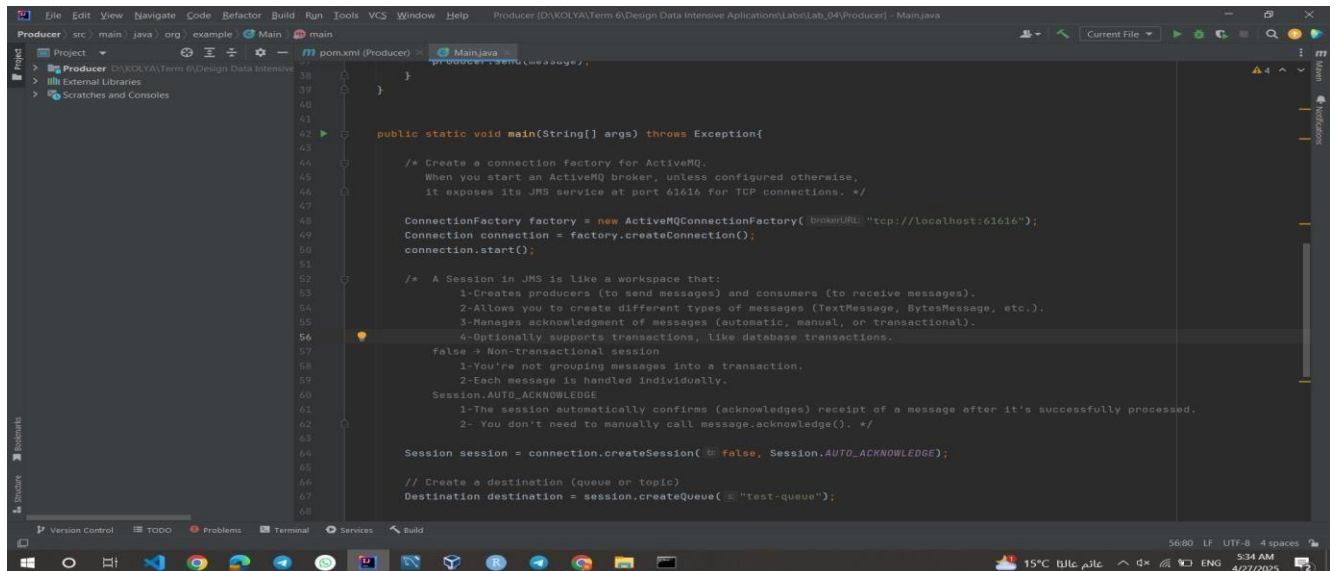
## - Notes:

- I try to increase the throughput as I can, but the system doesn't fail

## 2- Response time for both the produce and consume API calls

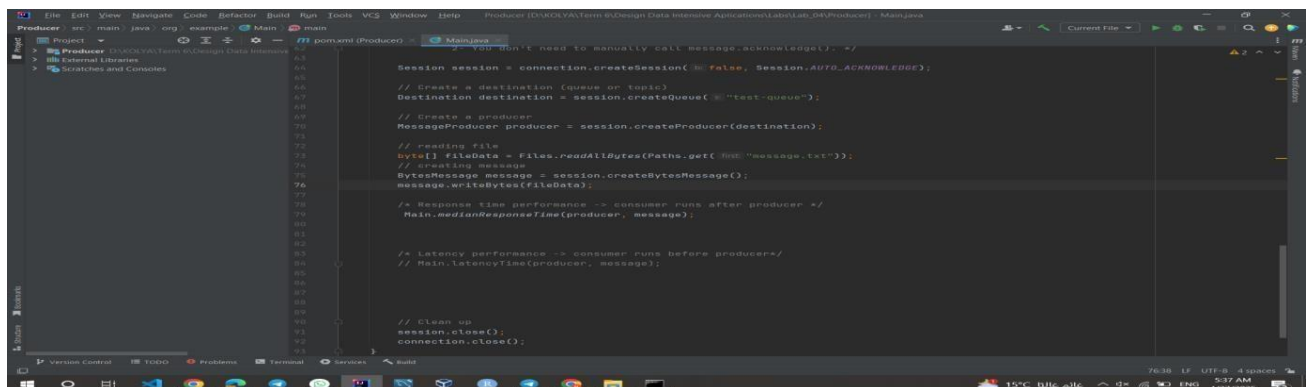
- **Producer**

Setup connection & session & queue



```
16 }
17
18 }
19
20 }
21
22 }
23
24 }
25
26 }
27
28 }
29
30 }
31
32 }
33
34 }
35
36 }
37
38 }
39
40 }
41
42 public static void main(String[] args) throws Exception{
43
44     /* Create a connection factory for ActiveMQ.
45      When you start an ActiveMQ broker, unless configured otherwise,
46      it exposes its JMS service at port 61616 for TCP connections. */
47
48     ConnectionFactory factory = new ActiveMQConnectionFactory( "tcp://localhost:61616");
49     Connection connection = factory.createConnection();
50     connection.start();
51
52     /* A Session in JMS is like a workspace that:
53      1-Creates producers (to send messages) and consumers (to receive messages).
54      2-Allows you to create different types of messages (TextMessage, BytesMessage, etc.).
55      3-Manages acknowledgment of messages (automatic, manual, or transactional).
56      4-Optionally supports transactions, like database transactions.
57      false -> Non-transactional session
58      1-You're not grouping messages into a transaction.
59      2-Each message is handled individually.
60      Session.AUTO_ACKNOWLEDGE
61      1-The session automatically confirms (acknowledges) receipt of a message after it's successfully processed.
62      2- You don't need to manually call message.acknowledge(). */
63
64     Session session = connection.createSession( false, Session.AUTO_ACKNOWLEDGE);
65
66     // Create a destination (queue or topic)
67     Destination destination = session.createQueue( "test-queue");
68
69 }
```

Creating producer & Preparing message & Calling medianResponseTime function.



```
63
64 Session session = connection.createSession( false, Session.AUTO_ACKNOWLEDGE);
65
66 // Create a destination (queue or topic)
67 Destination destination = session.createQueue( "test-queue");
68
69 // Create a producer
70 MessageProducer producer = session.createProducer(destination);
71
72 // reading file
73 byte[] fileData = Files.readAllBytes(Paths.get( "src/main/resources/message.txt"));
74 // creating message
75 BytesMessage message = session.createBytesMessage();
76 message.writeBytes(fileData);
77
78 /* Response time performance -> consumer runs after producer */
79 Main.medianResponseTime(producer, message);
80
81
82 /* Latency performance -> consumer runs before producers */
83 // Main.latencyTime(producer, message);
84
85
86
87
88
89 // Close up
90 session.close();
91 connection.close();
92 }
```

Function for testing producer Response Time.

The screenshot shows an IDE with a Java file named `main.java` in the `src` directory. The code is as follows:

```

1  import java.nio.file.Files;
2  import java.nio.file.Paths;
3  import java.util.Arrays;
4  import java.util.Date;
5
6
7
8
9
10
11 public class Main {
12
13     /* calculating the median & average for response time of 1000 sent messages*/
14     /* Usage */
15     public static void medianResponseTime(MessageProducer producer, ByteMessage message) throws Exception{
16
17         long[] elapsedTime = new long [1000] ;
18
19         for (int i=0; i<1000; i++) {
20             long start = System.currentTimeMillis();
21             producer.send(message);
22             elapsedTime[i] = System.currentTimeMillis() - start;
23         }
24
25         Arrays.sort(elapsedTime);
26         long median = (elapsedTime[499]+elapsedTime[500]) / 2 ;
27         double average = (Arrays.stream(elapsedTime).sum()) / 1000.0 ;
28         System.out.println("Median = " + median + " ms.");
29         System.out.println("Average = " + average + " ms.");
30     }
31
32     /* sending 10000 messages with time for latency*/
33     /* Usage */
34     public static void latencyTime(MessageProducer producer, ByteMessage message) throws Exception{
35         for(int i=0; i<10000; i++) {
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

```

Estimated producer response time

```

import java.time.Duration;
import java.util.concurrent.TimeUnit;
import org.apache.kafka.clients.producer.*;

public class Main {
    // Calculating the average response time for response time of 1000 sent messages
    public static void main(ResponseTime(MessageProducer producer, ByteArrayMessage) throws Exception {
        long[] elapsedTime = new long[1000];

        for (int i = 0; i < 1000; i++) {
            long start = System.currentTimeMillis();
            producer.sendMessage();
            elapsedTime[i] = System.currentTimeMillis() - start;
        }

        // Calculating the average response time
        long sum = 0;
        for (long time : elapsedTime) {
            sum += time;
        }
        long avgTime = sum / 1000;
        System.out.println("Average response time: " + avgTime + " ms");
    }
}

```

Process finished with exit code 0

Median : 2 ms.

Average : 2.42 ms.

- **Consumer**

Same setup as producer.

Function for testing consumer response time.

The screenshot shows an IDE window titled "Consumer - src/main/java/org/example/kafka/Main.java". The code defines a class `Main` with a static method `medianResponseTime` that calculates the median response time from 1000 messages. It uses `System.currentTimeMillis()` to measure time and sorts the resulting array.

```

import org.apache.kafka.clients.consumer.Consumer;
import java.util.Arrays;

public class Main {

    /* calculating the median & average for response time of 1000 received messages */
    public static void medianResponseTime(MessageConsumer consumer) throws Exception {

        long[] elapsedTime = new long[1000];

        for (int i = 0; i < 1000; i++) {
            long start = System.currentTimeMillis();
            Message message = consumer.receive();
            elapsedTime[i] = System.currentTimeMillis() - start;
        }

        Arrays.sort(elapsedTime);

        long median = (elapsedTime[499] + elapsedTime[500]) / 2;
        double average = (Arrays.stream(elapsedTime).sum()) / 1000.0;

        System.out.println("Median : " + median + " ms.");
        System.out.println("Average : " + average + " ms.");
    }
}

```

The bottom panel shows the execution results:

```

Run: Main
- Median : 0 ms.
- Average : 7.271 ms.

Process finished with exit code 0

```

Estimated consumer response time



Median : 0 ms.

Average : 7.271 ms.

## **B) Usability:**

### **1. Tool Setup Overhead**

- **Steps:**
  1. Download & extract ActiveMQ.
  2. Start broker (./bin/activemq start).
  3. Add Maven dependencies (activemq-client).
- **Time:** ~1.5 hours (first-time setup).

### **2. Code Cluttering**

- **Producer:**
  - **Lines:** 35.
  - **API Calls:** 7  
(ConnectionFactory → Connection → start() → Session → Queue → Producer → send()).
- **Consumer:**
  - **Lines:** 40.
  - **API Calls:** 7  
(ConnectionFactory → Connection → start() → Session → Queue → Consumer → receive()).

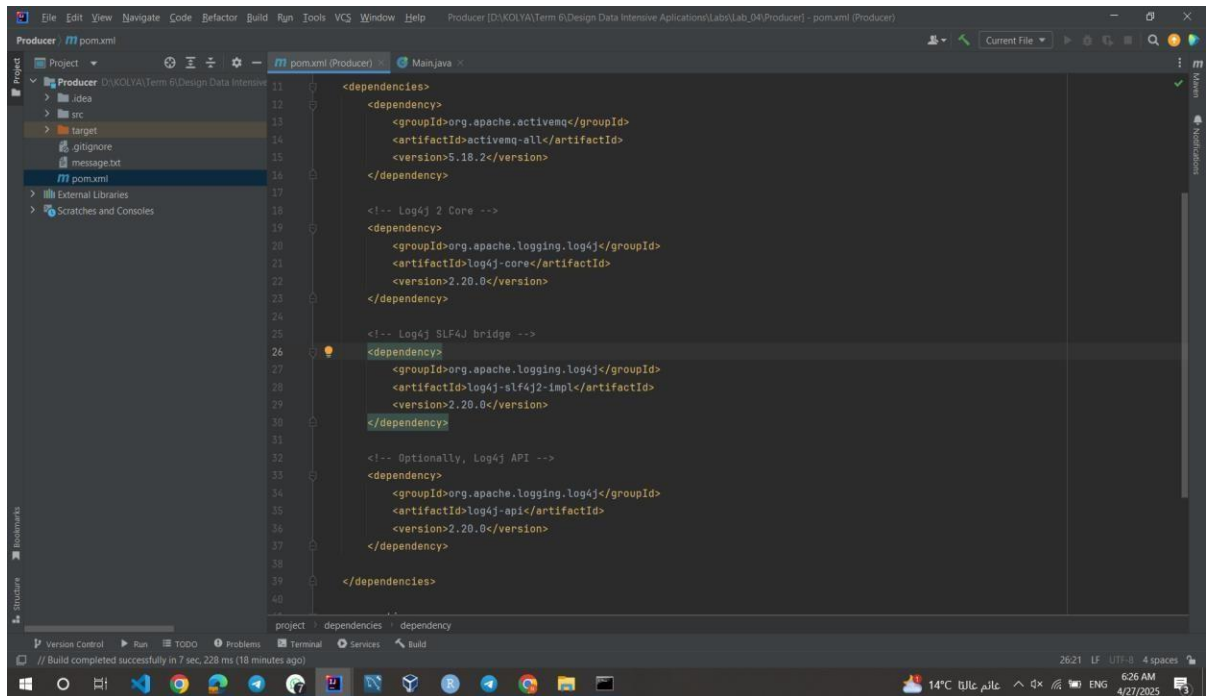
### **3. Pain Points**

1. Verbose API (7 APIs per operation).
2. Manual resource cleanup (close() calls).
3. Thread-unsafe Connection/Session.

- **ActiveMQ:**



- Took around 1 hour to install and learn.
- Install it from the website and extract it.
- Go to the bin file and open cmd and start the serve using “**activemq start**”
- **JMS with java:**
  - Took around 2-3 hours to use for first time.
  - Adding required dependencies (few dependencies are required), it take less than 1 hour to handle it :



- Few lines of code : less than 10 lines for settingup connection, session& queue , creating producer & consumer, clearing connection & session.
- API call is only one line of code.

### C) Integration:

- The JMS (Java Message Service) API itself is Java-only. BUT the *concepts* behind JMS (messaging, queues, topics, pub/sub systems) are not limited to Java.
- There are many message brokers (like ActiveMQ, RabbitMQ, Kafka, Amazon SQS, etc.) that speak JMS on one side for Java apps, but also provide clients for other languages. Messaging systems that *support* JMS may offer libraries for other languages too, just not using the JMS API exactly.
- It's not easy to integrate with other systems because of :
  1. JMS is a *messaging layer*, not a storage layer — it's not designed to dump messages straight into other systems.
  2. You usually need a small program that reads from JMS (consume messages), transforms the message if needed (parsing, formatting) for required system format saves the data into HDFS, Hive, Cassandra, etc.

---

## ☺ Kafka vs. JMS: Comparison Summary

### Apache Kafka

#### Advantages

- **High Throughput & Scalability:** Kafka handles millions of messages per second with horizontal scalability.
- **Durability & Fault Tolerance:** Uses distributed logs and replication.
- **Replayability:** Consumers can replay messages from any offset.
- **Decoupled Consumers:** Multiple consumer groups can independently process data.
- **Efficient Batch Processing:** Supports batching and zero-copy for better performance.

#### + Disadvantages

- **Complex Setup:** Requires Zookeeper (or KRaft mode), brokers, and topic configuration.
- **Message Ordering Limited:** Ordering guaranteed only within a partition.
- **High Learning Curve:** Needs understanding of topics, partitions, offsets.

---

### Java Message Service (JMS)

#### Advantages

- **Simplicity:** Easier to set up and use, especially with Java EE support.
- **Built-in Reliability:** Supports transactions and message acknowledgments.
- **Point-to-Point & Pub/Sub:** Native support for both messaging models.
- **Vendor Flexibility:** Works with many providers (ActiveMQ, IBM MQ, etc.).

#### + Disadvantages

- **Limited Scalability:** Not ideal for high-throughput distributed systems.
  - **Tight Coupling:** Often less flexible than Kafka in decoupling producers/consumers.
  - **No Message Replay:** Once consumed, messages can't be re-read by default.
-

### **Conclusion & Recommendation**

Based on scalability, performance, and flexibility, **Apache Kafka** is the preferred tool for modern, large-scale, and distributed systems. It excels in event streaming, real-time analytics, and microservices architecture.

However, if your system requires simpler messaging with strong reliability guarantees and low complexity (e.g., traditional enterprise applications), **JMS** might be more appropriate.

#### **Recommendation:**

Use **Kafka** when building scalable, fault-tolerant systems or handling large volumes of data. Use **JMS** for simpler, smaller-scale enterprise messaging where message durability and simplicity are prioritized.